Online Judge Platform v2.0

Software Requirements Specification (SRS)

Author

Sai Manish Reddy Ambati (IMT2022047)

Institute

International Institute of Information Technology, Bangalore

March 13, 2026

Contents

1 Introduction **3**

1.1 Purpose . . . 3

1.2 Scope . . . 3

1.3 Target Users . . . 4

1.4 Definitions, Abbreviations, and Acronyms . . . 4

2 Overall System Description **4**

2.1 Product Perspective . . . 4

2.2 Technology Stack . . . 5

2.3 User Classes and Characteristics . . . 5

2.4 Operating Environment . . . 6

2.5 Constraints . . . 6

2.6 Assumptions and Dependencies . . . 7

3 Functional Requirements **8**

3.1 Authentication and Authorization . . . 9

3.2 Problem Management . . . 9

3.3 Contest Management . . . 9

3.4 Code Submission and Judging . . . 10

3.5 Caching . . . 10

3.6 AI Features (Stub Implementation) . . . 10

3.7 Functional Module Mapping . . . 11

4 Non-Functional Requirements **11**

4.1 Performance . . . 11

4.2 Security . . . 12

4.3 Reliability . . . 12

4.4 Scalability . . . 12

4.5 Maintainability . . . 13

4.6 Logging and Portability . . . 13

5 System Architecture and Design **14**

5.1 Architecture Overview . . . 14

5.2 Design Patterns Applied . . . 15

5.3 SOLID Principles Applied . . . 16

5.4 Microservices Overview . . . 16

5.5 Data Architecture . . . 17

5.6 Caching Strategy . . . 18

5.7 AI Assistant Architecture (Stub) . . . 18

5.8 Deployment Architecture . . . 19

6 DevOps Practices and CI/CD **20**
6.1 CI/CD Pipeline . . . 20
6.2 Monitoring and Logging . . . 20

7 Security and Isolation **21**
7.1 Code Execution Security . . . 21
7.2 Authentication and Authorization Security . . . 21
7.3 API Security . . . 21
7.4 Data Protection . . . 21

8 Future Enhancements (Out of Scope for v2.0) **22**
8.1 Production-Grade AI Assistant . . . 22
8.2 Learning and Collaboration . . . 22
8.3 Platform Expansion . . . 22

9 Glossary and References **22**
9.1 Glossary . . . 22

10 Appendix **23**
10.1 API Endpoint Summary . . . 23
10.1.1 Authentication . . . 23
10.1.2 Problems . . . 23
10.1.3 Submissions . . . 23
10.1.4 AI Assistant (Stub) . . . 24
10.1.5 Contests and Leaderboard . . . 24
10.1.6 Administration . . . 24
10.2 Technology Versions . . . 25

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the **Online Judge Platform v2.0** — a web-based competitive programming system supporting problem solving, contest participation, automated code evaluation, and a lightweight AI-assisted code analysis feature.

The document reflects the actual implemented system, grounded in a clean layered microservices architecture and industry-standard design patterns.

The system is designed to:

- Provide a scalable, modular environment for practicing algorithmic programming
- Automatically evaluate code submissions across multiple languages (C++, Python, Java)
- Support structured programming contests with real-time leaderboards
- Expose stub AI endpoints (free-tier API calls) for code explanation and debugging
- Enable academic institutions to conduct programming assessments

1.2 Scope

The current version of the platform includes the following core capabilities.

Included Features

- User authentication with JWT and role-based access control (RBAC)
- Problem repository with full CRUD operations (create/update by `problem_setter`; delete by `admin` only)
- Multi-language code submission, compilation, and sandboxed execution
- Real-time verdict generation (AC, WA, TLE, RE, CE)
- Redis-backed caching with automatic cache invalidation
- Contest management with participant registration and leaderboards
- Stub AI assistant endpoints (explanation and debugging) using free-tier LLM API calls
- Graceful degradation when Redis is unavailable

Out of Scope (Future Enhancements)

- Video tutorials and editorial content
- Live coding interview features
- Peer code review systems
- Native mobile applications
- Full production-grade AI with rate limiting, caching, and cost optimization

1.3 Target Users

The system supports the following user roles.

Role	Description
Student / User	Solves problems, participates in contests, and may use AI hints for learning.
Problem Setter	Creates and manages problems, including problem statements and test cases.
Contest Organizer	Configures contests, schedules events, and manages contest participants.
Administrator	Manages users, system roles, and platform-wide operations.
Guest (Unauthenticated)	Can browse and filter problems but cannot submit solutions.

Table 1: Target User Roles

1.4 Definitions, Abbreviations, and Acronyms

Term	Definition
OJ	Online Judge — automated code evaluation system
RBAC	Role-Based Access Control
JWT	JSON Web Token used for stateless authentication
DTO	Data Transfer Object used to validate incoming request data
MERN	MongoDB, Express.js, React.js, Node.js technology stack
REST	Representational State Transfer architecture
AC	Accepted submission
WA	Wrong Answer
TLE	Time Limit Exceeded
RE	Runtime Error
CE	Compilation Error
LLM	Large Language Model
CI/CD	Continuous Integration / Continuous Deployment
SRP / OCP / DIP	SOLID design principles (Single Responsibility, Open/Closed, Dependency Inversion)

Table 2: Definitions and Acronyms

2 Overall System Description

2.1 Product Perspective

The Online Judge Platform v2.0 is a distributed web application built using a clean layered microservices architecture. The system is split into two independent services — an **API Service** and an **Execution Service** — each following the layered pattern:

```
Route → Controller → Service → Repository → Model
```

Both services are stateless and communicate through well-defined REST interfaces, with an optional message queue integration path already architecturally planned.

This separation ensures clear responsibility boundaries, improved maintainability, and easier scalability for code submission processing.

2.2 Technology Stack

Layer	Technology	Purpose
Frontend Framework	React.js	Component-based user interface and client-side state management
Frontend Hosting	Vercel	Static React build hosted on CDN with automatic deployments
Backend Framework	Node.js + Express.js	RESTful API services and business logic implementation
Backend Hosting	Render / Railway	Managed Node.js hosting with Redis add-on support
Execution Hosting	AWS EC2 (Free Tier)	Compiler and judging worker deployed on `t2.micro` or `t3.micro` instance
Database	MongoDB (Atlas or local)	Persistent NoSQL storage for users, problems, submissions, and contests
Caching	Redis	Managed Redis add-on on Render/Railway (production); local Redis during development
Containerization	Docker (optional)	Used on EC2 for isolated execution environments; not required for local development
AI Integration	OpenAI / Anthropic API	Stub AI endpoints for code explanation and debugging (free-tier usage)

Table 3: Technology Stack

2.3 User Classes and Characteristics

User Class	Characteristics	Key Requirements
Student	Beginner to advanced programmers	Problem solving, contest participation, optional AI hints
Problem Setter	Experienced programmers or educators	Problem CRUD operations and test case management
Contest Organizer	Academic or competitive programming mentors	Contest setup and leaderboard access
Administrator	System maintainers	Full user and system management
Guest	Unauthenticated visitors	Read-only access to problem list

Table 4: User Classes and Characteristics

2.4 Operating Environment

Client-Side Requirements

- Modern web browsers with JavaScript enabled
- Minimum screen resolution: 1024×768 with a stable internet connection

Server-Side Environment

- Frontend served via **Vercel** (CDN-backed static hosting with automatic deployments from Git)
- API Service deployed on **Render or Railway** with managed Redis add-on for caching
- Execution Service deployed on an **AWS EC2 free-tier instance** (`t2.micro`/`t3.micro`) responsible for compilation and sandboxed code execution
- MongoDB hosted on **MongoDB Atlas** free tier
- Redis provided as a managed add-on by Render/Railway; the system degrades gracefully if Redis becomes unavailable
- No Docker or Kubernetes required for local development

2.5 Constraints

Technical Constraints

- Code execution timeout: 10 seconds per submission
- Compilation timeout: 30 seconds
- Maximum code file size restrictions enforced at the API boundary
- AI API usage governed by free-tier rate limits
- MongoDB connection pooling limits under concurrent load

Infrastructure Constraints

- EC2 free-tier instance (`t2.micro`) has limited CPU and RAM, restricting concurrent judging capacity
- Horizontal scaling of judge workers is deferred to future versions
- Render/Railway free tiers may introduce cold-start latency for idle backend services
- Vercel free tier has serverless function timeouts (not applicable since the frontend is static)

Regulatory Constraints

- Protection of user data and credentials
- Intellectual property ownership of submitted code
- Responsible use of AI-generated responses with clear labeling

2.6 Assumptions and Dependencies

Assumptions

- Users have reliable internet connectivity
- MongoDB is accessible (local instance or MongoDB Atlas)
- AI API keys are configured in the environment configuration file (`.env`)
- Free-tier API quotas are sufficient for demonstration purposes
- Problem test cases are correctly defined by problem setters

External Dependencies

- MongoDB (local instance or MongoDB Atlas)
- OpenAI or Anthropic API for stub AI features
- Node.js ecosystem libraries including Express, Mongoose, bcrypt, jsonwebtoken, and ioredis

3 Functional Requirements

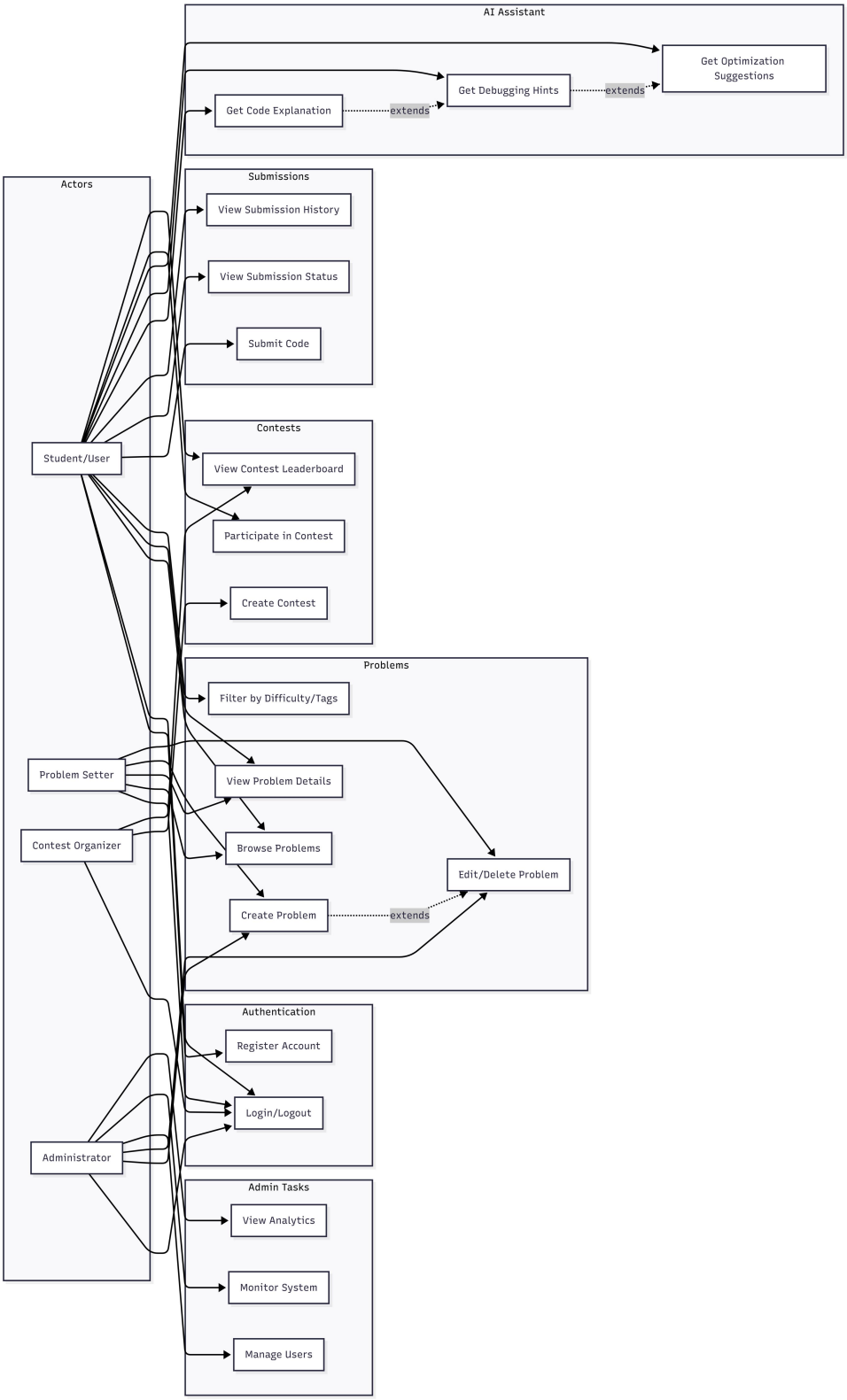

Figure 1: Use Case Diagram of the Online Judge Platform

3.1 Authentication and Authorization

ID	Requirement	Priority
FR-01	The system shall allow users to register with a unique username, email, and password. Input validation is handled via `RegisterUserDTO`; passwords are hashed with bcrypt (12 salt rounds).	High
FR-02	The system shall authenticate users via JWT tokens and enforce role-based access control (RBAC) for roles: student, problem_setter, admin, and guest. Middleware components enforce route-level permissions.	High
FR-03	All passwords shall be hashed with bcrypt before storage. Plaintext passwords shall never appear in API responses or logs.	High

Table 5: Authentication and Authorization Requirements

3.2 Problem Management

ID	Requirement	Priority
FR-04	The system shall allow `problem_setter` and `admin` roles to create and update problems. Only the administrator may soft-delete a problem using an `is_deleted` flag.	High
FR-05	Each problem shall contain title, description, difficulty, tags, constraints, visible and hidden test cases, time and memory limits, and multi-language boilerplate code (C++, Python, Java).	High
FR-06	Any user, including unauthenticated guests, shall be able to view and filter the problem list with pagination. Results may be served from Redis cache if available.	High

Table 6: Problem Management Requirements

3.3 Contest Management

ID	Requirement	Priority
FR-07	The system shall allow administrators and problem setters to create contests specifying title, problem list, start time, and end time. Input validation is performed using `CreateContestDTO`.	Medium
FR-08	The system shall validate that all problem IDs in a contest exist and prevent duplicate problem entries in the contest configuration.	Medium
FR-09	Students shall be able to register for contests. Duplicate registrations shall be rejected with an appropriate error response. Leaderboards display real-time rankings.	Medium

Table 7: Contest Management Requirements

3.4 Code Submission and Judging

ID	Requirement	Priority
FR-10	The system shall accept code submissions in C++, Python, and Java via `SubmitCodeDTO`. Each submission is compiled and executed against all test cases in a sandboxed environment.	High
FR-11	The system shall produce verdicts including Accepted (AC), Wrong Answer (WA), Time Limit Exceeded (TLE), Runtime Error (RE), and Compilation Error (CE).	High
FR-12	The system shall record execution time and memory usage for each submission. Users may query submission history and status filtered by problem or user.	High

Table 8: Code Submission and Judging Requirements

3.5 Caching

ID	Requirement	Priority
FR-13	The system shall cache problem lists, problem details, and contest lists using Redis. Cache entries are invalidated on create, update, or delete operations. The system shall continue operating if Redis becomes unavailable (cache-aside strategy with graceful degradation).	High

Table 9: Caching Requirements

3.6 AI Features (Stub Implementation)

ID	Requirement	Priority
FR-14	The system shall expose REST endpoints for AI-powered code explanation (`/api/ai/explain`), debugging assistance (`/api/ai/debug`), and optimization suggestions (`/api/ai/optimize`). These endpoints make free-tier API calls to an external language model.	Medium

Table 10: AI Feature Requirements

3.7 Functional Module Mapping

Functional Area	Service	Key Modules
Authentication (FR-01 to FR-03)	API Service	AuthController → AuthService → UserRepository → User Model
Problem Management (FR-04 to FR-06)	API Service	ProblemController → ProblemService → ProblemRepository → Problem Model
Contest Management (FR-07 to FR-09)	API Service	ContestController → ContestService → ContestRepository → Contest Model
Code Submission & Judging (FR-10 to FR-12)	Execution Service	SubmissionController → SubmissionService → JudgeService → ExecutorFactory → SubmissionRepository
Caching (FR-13)	API Service	RedisClient integrated into ProblemService and ContestService
AI Features (FR-14)	Execution Service	AIService (stub implementation using free-tier LLM calls)
RBAC Enforcement	API Service	AuthMiddleware and RBACMiddleware
Error Handling	Shared	AppError factory methods and centralized ErrorMiddleware
Logging	Shared	Logger with timestamped log levels and module context

Table 11: Functional Module Mapping

4 Non-Functional Requirements

4.1 Performance

ID	Requirement
NFR-01	Cached API responses (problem lists, contest lists) shall be served within 50ms. Non-cached database-backed responses shall complete within 200ms under normal load.
NFR-02	Code execution shall timeout after 10 seconds per submission. Compilation shall timeout after 30 seconds. Submissions exceeding these limits receive a Time Limit Exceeded (TLE) or Compilation Error (CE) verdict.

Table 12: Performance Requirements

4.2 Security

ID	Requirement
NFR-03	User passwords shall be hashed with bcrypt using 12 salt rounds. JWT tokens expire after a configurable period (default: 1 day). Plaintext passwords shall never appear in API responses, logs, or error messages.
NFR-04	All incoming request data shall be validated via DTO classes before reaching the service layer. Role-based access is enforced declaratively through RBACMiddleware on protected routes.

Table 13: Security Requirements

4.3 Reliability

ID	Requirement
NFR-05	The system shall degrade gracefully when Redis is unavailable. All functionality continues without caching. Redis failures are logged but do not propagate exceptions to the client.
NFR-06	All unhandled errors shall be caught by centralized ErrorMiddleware. Internal error details such as stack traces or database query details shall not be exposed in API responses.

Table 14: Reliability Requirements

4.4 Scalability

ID	Requirement
NFR-07	The API Service and Execution Service shall run as independent Node.js processes and are architecturally ready for message queue integration (e.g., BullMQ or AWS SQS) to support asynchronous submission processing.
NFR-08	New programming languages shall be addable by implementing a new subclass of `BaseExecutor` and registering it in `ExecutorFactory`, without modifying `JudgeService` or existing executor classes (Open/Closed Principle).

Table 15: Scalability Requirements

4.5 Maintainability

ID	Requirement
NFR-09	The codebase follows a clean layered architecture: Route → Controller → Service → Repository → Model. Controllers handle only HTTP request/response logic while all business logic resides in the service layer.
NFR-10	All configuration values (secrets, database URIs, API keys, token expiry) are externalized to `.env` files and accessed through a centralized `AppConfig` singleton. Business logic shall not contain hardcoded configuration values.

Table 16: Maintainability Requirements

4.6 Logging and Portability

ID	Requirement
NFR-11	All key system events (requests, errors, cache hits/misses, verdict outcomes) shall be logged with timestamp, severity level, and module context using a shared Logger class.
NFR-12	The system shall run locally with Node.js, MongoDB, and optionally Redis. Docker or Kubernetes are not required for development and testing, though container deployment remains supported.

Table 17: Logging and Portability Requirements

5 System Architecture and Design

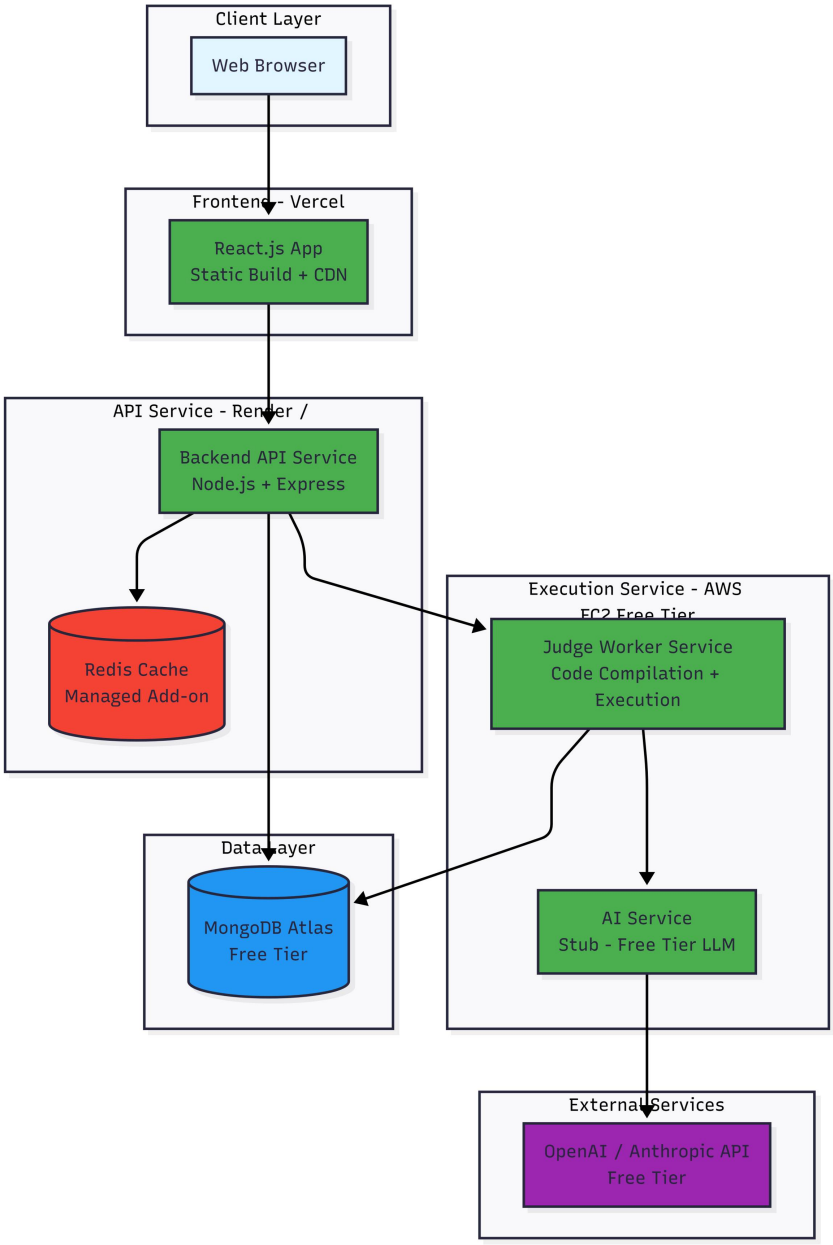

Figure 2: High-Level System Architecture

5.1 Architecture Overview

The Online Judge Platform v2.0 follows a clean layered microservices architecture organized into two independent services: the **API Service** and the **Execution Service**. The API Service handles authentication, problem management, contest management, and caching, while the Execution Service manages code submission processing, judging, and AI-assisted features.

Both services share a common utility layer containing components such as `AppConfig`, `Logger`, `AppError`, and `ErrorMiddleware`, extracted into a shared module.

Request flow inside each service follows a strict layered architecture:

```
HTTP Request → Route → Controller → Service → Repository → MongoDB
```

Key architectural characteristics:

- Service-oriented separation between API and Execution services
- Stateless backend services with authentication managed through JWT tokens
- Shared persistent state stored in MongoDB and Redis
- Asynchronous judging pipeline designed for message queue integration
- Cache-aside pattern using Redis for frequently accessed data
- Graceful degradation when optional infrastructure such as Redis becomes unavailable

5.2 Design Patterns Applied

Pattern	Where Applied	Purpose
Repository Pattern	UserRepository, ProblemRepository, ContestRepository, SubmissionRepository	Decouples business logic from database queries and allows future data source substitution.
Factory Pattern	ServiceFactory	Centralized dependency injection for creating services, controllers, and route dependencies.
Strategy Pattern	BaseExecutor → CppExecutor, PythonExecutor + ExecutorFactory	Enables support for new programming languages without modifying existing judging logic.
Singleton Pattern	AppConfig, ServiceFactory	Ensures a single shared instance of configuration and service objects.
Cache-Aside Pattern	ProblemService, ContestService	Checks cache before database access and invalidates cache entries on updates.
DTO Pattern	RegisterUserDTO, LoginDTO, CreateProblemDTO, CreateContestDTO, SubmitCodeDTO	Validates and structures incoming request data before reaching the service layer.
Middleware Pattern	AuthMiddleware, RBACMiddleware, ErrorMiddleware	Applies cross-cutting concerns such as authentication and error handling declaratively.

Table 18: Design Patterns Used

5.3 SOLID Principles Applied

Principle	Application in System
Single Responsibility (SRP)	Controllers handle HTTP communication, services contain business logic, repositories manage database access, and DTOs validate incoming requests.
Open/Closed (OCP)	New programming languages can be supported by implementing additional `BaseExecutor` subclasses without modifying existing services.
Liskov Substitution (LSP)	Executor subclasses such as `CppExecutor` and `PythonExecutor` can replace `BaseExecutor` without altering program behavior.
Interface Segregation (ISP)	Repositories expose only necessary database operations required by their consumers.
Dependency Inversion (DIP)	Services depend on repository abstractions injected through `ServiceFactory` rather than directly on database implementations.

Table 19: SOLID Principles in the Architecture

5.4 Microservices Overview

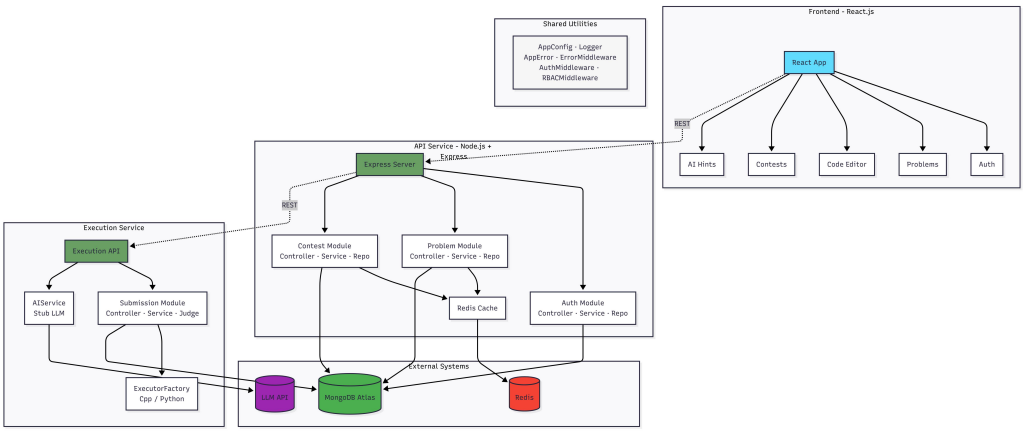

Figure 3: Component Diagram of the Platform Services

API Service

Handles authentication, problem management, contest management, and caching. It exposes REST endpoints consumed by the frontend and integrates with MongoDB and Redis for persistence and caching.

Execution Service

Processes code submissions asynchronously. It uses `ExecutorFactory` to select the appropriate language executor (such as C++ or Python), compiles and runs submitted code within sandboxed environments, and records verdicts and execution metrics.

Shared Utilities

Shared utilities include:

- **AppConfig** — centralized configuration manager

- **Logger** — structured timestamped logging utility
- **AppError** — structured application error abstraction
- **ErrorMiddleware** — centralized error handling mechanism

5.5 Data Architecture

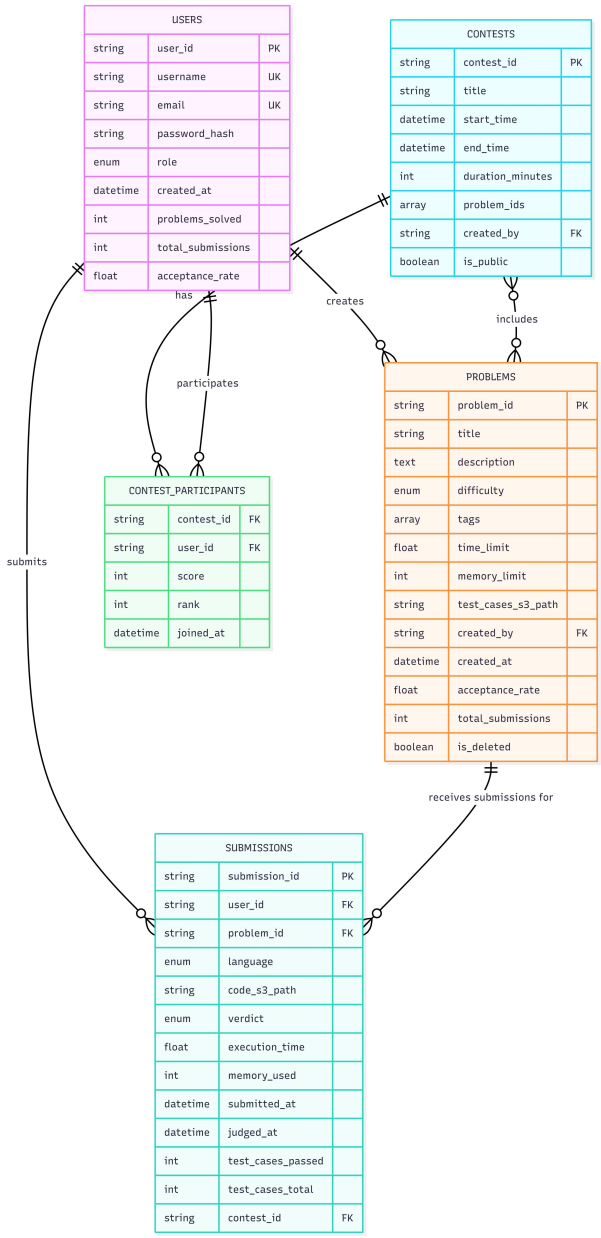

Figure 4: Entity Relationship Diagram

The system uses MongoDB with Mongoose schemas for persistent storage. Key entity relationships include:

- Users have many submissions; each submission belongs to one user and one problem
- Problems belong to a creator with the `problem_setter` or `admin` role

- Contests contain lists of problem IDs and participant records
- Submissions may optionally reference a contest ID for contest-mode evaluation

Data design principles:

- Indexed fields such as `user_id`, `problem_id`, and `contest_id` support fast filtering and sorting
- Soft-delete flags preserve historical data integrity
- Aggregated statistics such as acceptance rate and submission counts improve leaderboard performance

5.6 Caching Strategy

The platform implements a Redis-based cache-aside strategy for frequently accessed data such as problem lists and contest information.

1. The application first checks Redis for cached data.
2. On a cache miss, the system queries MongoDB and stores the result in Redis.
3. When data is created, updated, or deleted, related cache entries are invalidated.

If Redis becomes unavailable, the caching layer is bypassed and the system continues operating using MongoDB directly.

5.7 AI Assistant Architecture (Stub)

The AIService is a lightweight stub implementation demonstrating integration with large language model APIs. The service receives problem context and user code, sends a request to a free-tier LLM API (OpenAI or Anthropic), and returns structured responses for explanation, debugging, or optimization suggestions.

Current endpoints include:

- `POST /api/ai/explain`
- `POST /api/ai/debug`
- `POST /api/ai/optimize`

Production-grade features such as rate limiting, caching, circuit breakers, and cost management are planned as future enhancements and are intentionally outside the scope of version 2.0.

5.8 Deployment Architecture

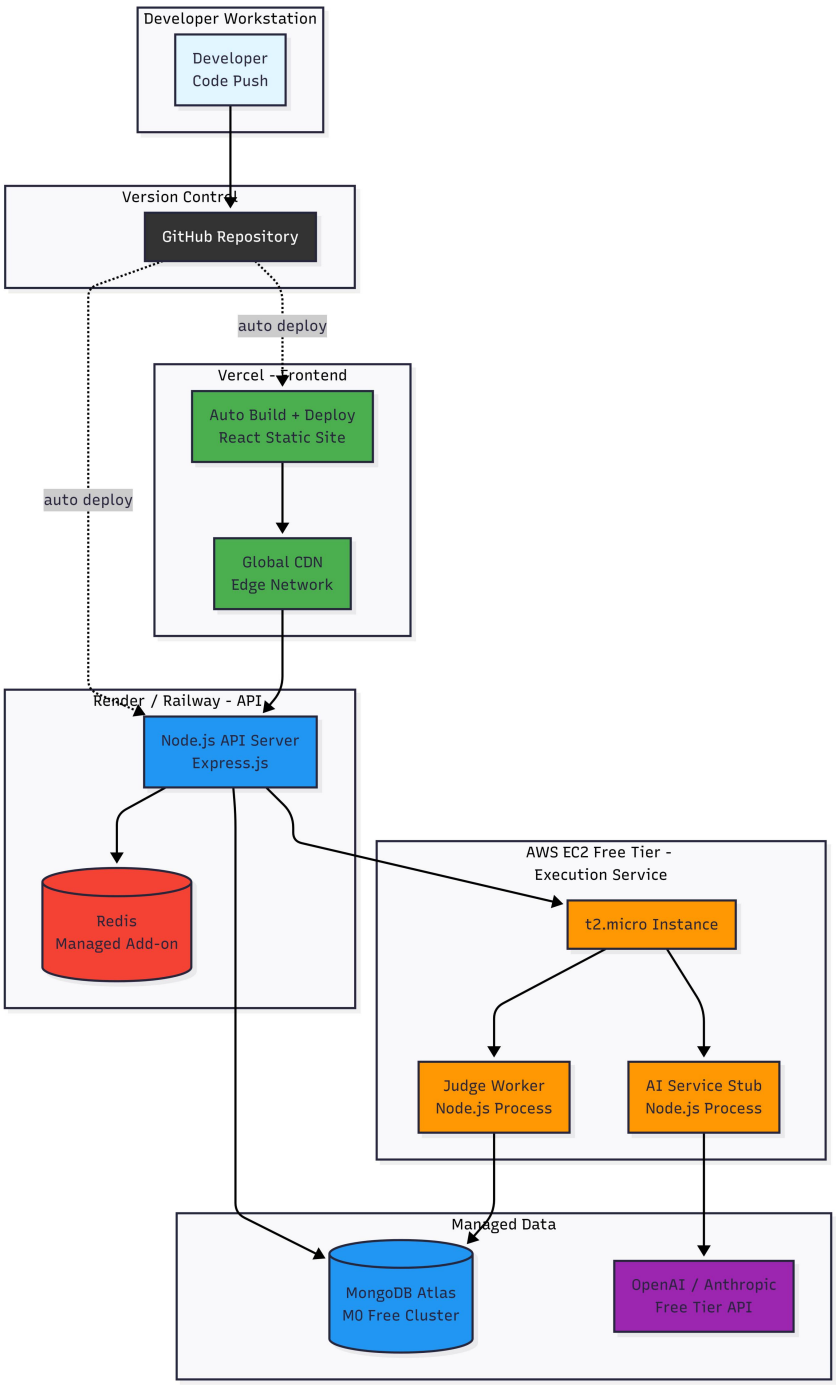

Figure 5: Deployment Architecture of the Online Judge Platform

- **Frontend**: Hosted on Vercel with CDN-backed static deployment and automatic Git-based previews.
- **API Service**: Deployed on Render or Railway with managed Node.js hosting and integrated Redis add-on.
- **Execution Service**: Hosted on an AWS EC2 free-tier instance providing full OS-level control for sandboxed code execution.

- **Database**: MongoDB Atlas free-tier cluster for persistent storage.
- **Service Communication**: All inter-service communication occurs over HTTPS using environment-configured endpoints.

6 DevOps Practices and CI/CD

6.1 CI/CD Pipeline

The project adopts a CI/CD pipeline to automate testing, building, and deployment. This ensures consistent releases and enables early detection of defects during development.

Pipeline Stages

- **Code Commit** — Developers push changes to the version control repository.
- **Automated Testing** — Unit tests, integration tests, and code quality checks (ESLint, coverage) are executed automatically.
- **Container Build** — Docker images are built and versioned for updated services.
- **Registry Push** — Built images are stored in a container registry and scanned for vulnerabilities.
- **Deployment** — Rolling updates deploy the new version to the target environment.
- **Health Verification** — Automated readiness checks confirm service availability.

6.2 Monitoring and Logging

Centralized Logging

All services use the shared `Logger` class to generate structured, timestamped logs with severity levels (INFO, WARN, ERROR) and module context. Logs are aggregated to support operational monitoring and debugging.

Metrics and Monitoring

Operational metrics monitored include:

- API request latency and error rates
- Judging queue depth and worker throughput
- Cache hit/miss ratios
- Service health and uptime status

Alerting

Automated alerts notify administrators when abnormal conditions occur, such as high error rates, resource failures, or unusual workload spikes. Notifications are delivered through configured communication channels.

7 Security and Isolation

7.1 Code Execution Security

Executing untrusted user code requires strict sandboxing. The Execution Service applies multiple isolation layers to ensure platform safety.

Container Isolation

Each submission runs inside an isolated execution environment that is destroyed after completion. CPU, memory, and runtime resource limits prevent denial-of-service attacks.

Network Isolation

Execution environments operate without external network access, preventing unauthorized communication with external systems.

Filesystem Restrictions

Execution environments allow minimal write access with temporary storage cleared after each run. Execution and compilation timeouts enforce strict resource limits.

7.2 Authentication and Authorization Security

Concern	Implementation
Credential Storage	bcrypt hashing with 12 salt rounds; plaintext passwords are never stored, logged, or returned in API responses.
Token-Based Authentication	JWT tokens with configurable expiry (default: 1 day). Tokens are validated on protected routes via `AuthMiddleware`.
Role Enforcement	Declarative RBAC using `RBACMiddleware`. Each route defines the minimum required role(s).
Input Validation	All request payloads validated through DTO classes before reaching the service layer.

Table 20: Authentication and Authorization Security

7.3 API Security

- All user inputs are validated and sanitized via DTO classes to prevent injection or scripting attacks.
- Rate limiting protects the system from abuse and excessive automated requests.
- All client-server communication occurs over encrypted HTTPS connections in production.
- Internal error details are never exposed in API responses; centralized `ErrorMiddleware` returns safe error objects.

7.4 Data Protection

- Sensitive data is encrypted during storage and transmission.
- System credentials and API keys are stored in `.env` files and injected at runtime rather than hardcoded.
- User data and submission records are protected by RBAC access controls.

8 Future Enhancements (Out of Scope for v2.0)

8.1 Production-Grade AI Assistant

- Per-user AI request rate limiting and cost tracking
- Response caching for common query patterns
- Circuit breakers and fallback responses for LLM API failures
- Automatic test case generation and plagiarism detection

8.2 Learning and Collaboration

- Video-based tutorials and guided solution walkthroughs
- Personalized problem recommendations based on user performance
- Peer code review and discussion forums
- Live coding interview sessions

8.3 Platform Expansion

- Native mobile applications for iOS and Android
- Multi-tenant institutional deployment
- Message queue integration (BullMQ or AWS SQS) for scalable asynchronous submission processing
- Social networking and user collaboration features

9 Glossary and References

9.1 Glossary

Term	Definition
Repository Pattern	Design pattern separating business logic from database access code.
Factory Pattern	Centralized object creation and dependency wiring.
Strategy Pattern	Family of interchangeable algorithms selectable at runtime (e.g., language executors).
Cache-Aside Pattern	Application-managed caching: read from cache first, populate on miss, invalidate on write.
DTO	Data Transfer Object used for validation and structuring of request data.
Soft Delete	Marking a record as deleted using a flag without physically removing it from the database.
Graceful Degradation	System continues functioning with reduced capability when an optional component fails.

Table 21: Glossary

10 Appendix

10.1 API Endpoint Summary

10.1.1 Authentication

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a new user account
POST	/api/auth/login	Public	Authenticate user and return JWT token
POST	/api/auth/logout	Authenticated	Invalidate user session

Table 22: Authentication Endpoints

10.1.2 Problems

Method	Endpoint	Access	Description
GET	/api/problems	Public	List problems with filters and pagination
GET	/api/problems/:id	Public	Retrieve full problem details
POST	/api/problems	problem_setter, admin	Create new problem
PUT	/api/problems/:id	problem_setter, admin	Update existing problem
DELETE	/api/problems/:id	admin only	Soft-delete problem

Table 23: Problem Endpoints

10.1.3 Submissions

Method	Endpoint	Access	Description
POST	/api/submissions	Authenticated	Submit code for evaluation
GET	/api/submissions/:id	Authenticated	Retrieve submission status and verdict
GET	/api/users/:user-name/submissions	Authenticated	View submission history for a user

Table 24: Submission Endpoints

10.1.4 AI Assistant (Stub)

Method	Endpoint	Access	Description
POST	/api/ai/explain	Authenticated	Get AI-generated code explanation
POST	/api/ai/debug	Authenticated	Receive debugging hints for submissions
POST	/api/ai/optimize	Authenticated	Get optimization suggestions

Table 25: AI Assistant Endpoints

10.1.5 Contests and Leaderboard

Method	Endpoint	Access	Description
GET	/api/contests	Public	List contests
POST	/api/contests	problem_setter, admin	Create contest
POST	/api/contests/:id/register	Authenticated student	Register for a contest
GET	/api/contests/:id/leaderboard	Authenticated	View contest leaderboard
GET	/api/leaderboard	Public	View global leaderboard

Table 26: Contest Endpoints

10.1.6 Administration

Method	Endpoint	Access	Description
GET	/api/admin/users	admin	List all users
PUT	/api/admin/users/:id/role	admin	Update user role
GET	/api/admin/metrics	admin	Retrieve system metrics

Table 27: Administration Endpoints

10.2 Technology Versions

Technology	Version
React.js	18.2.0
Node.js	20.x LTS
Express.js	4.18.x
MongoDB	7.0
Mongoose	8.x
Redis	7.x
Docker	24.x
Kubernetes	1.28+

Table 28: Technology Versions